

Evaluating a microservice architecture

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

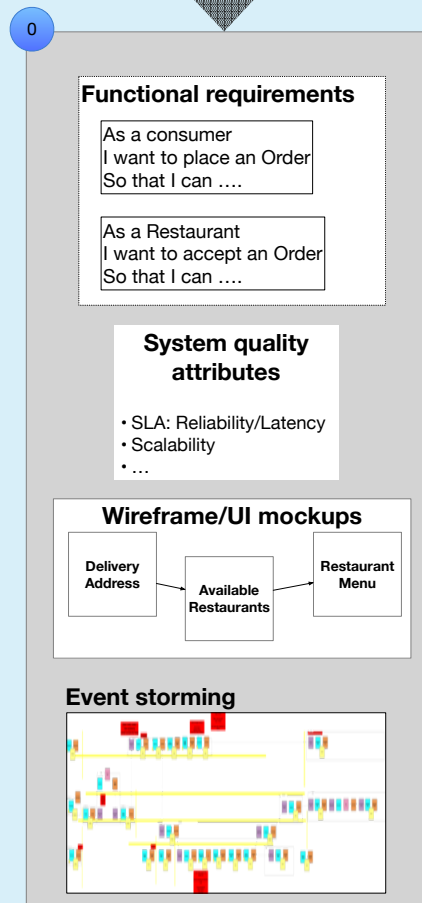
Author of POJOs in Action and Microservices Patterns

 @crichardson

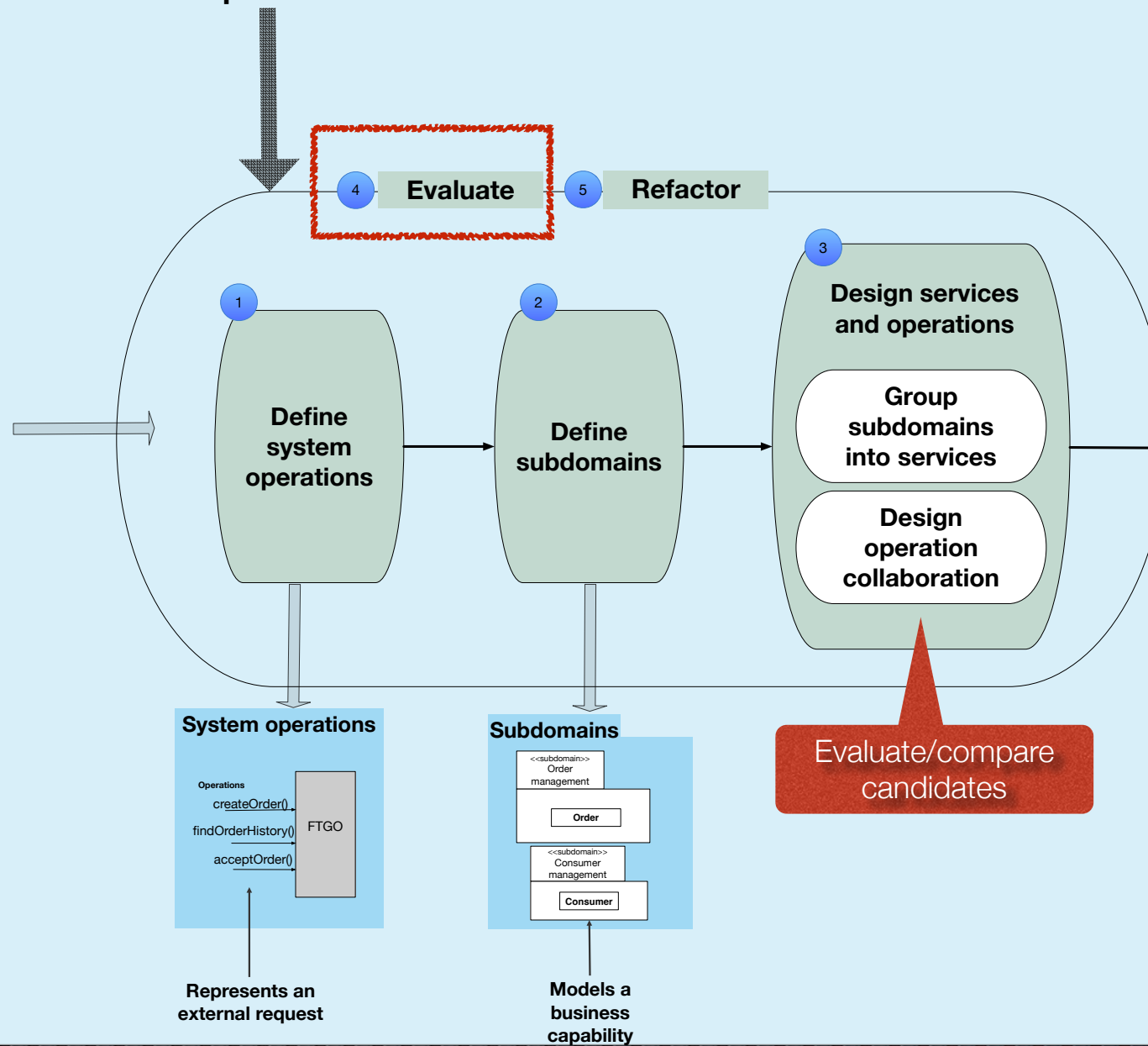
chris@chrisrichardson.net

adopt.microservices.io

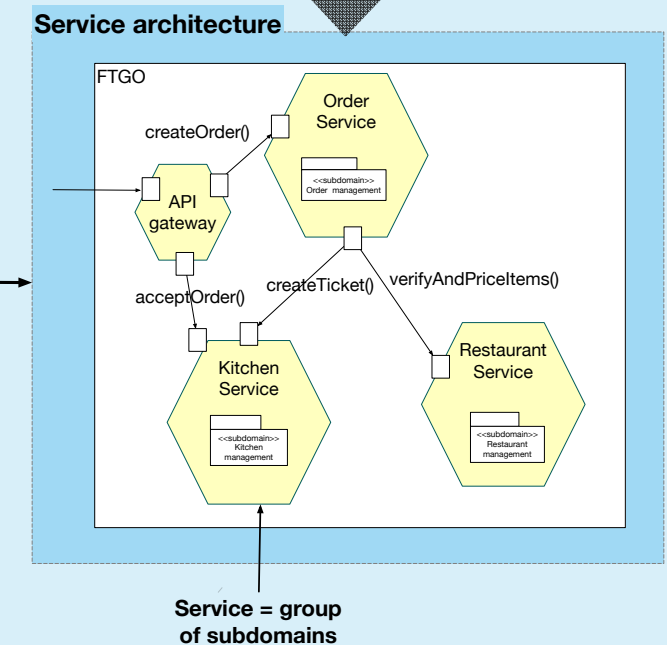
The starting point



The process



The result

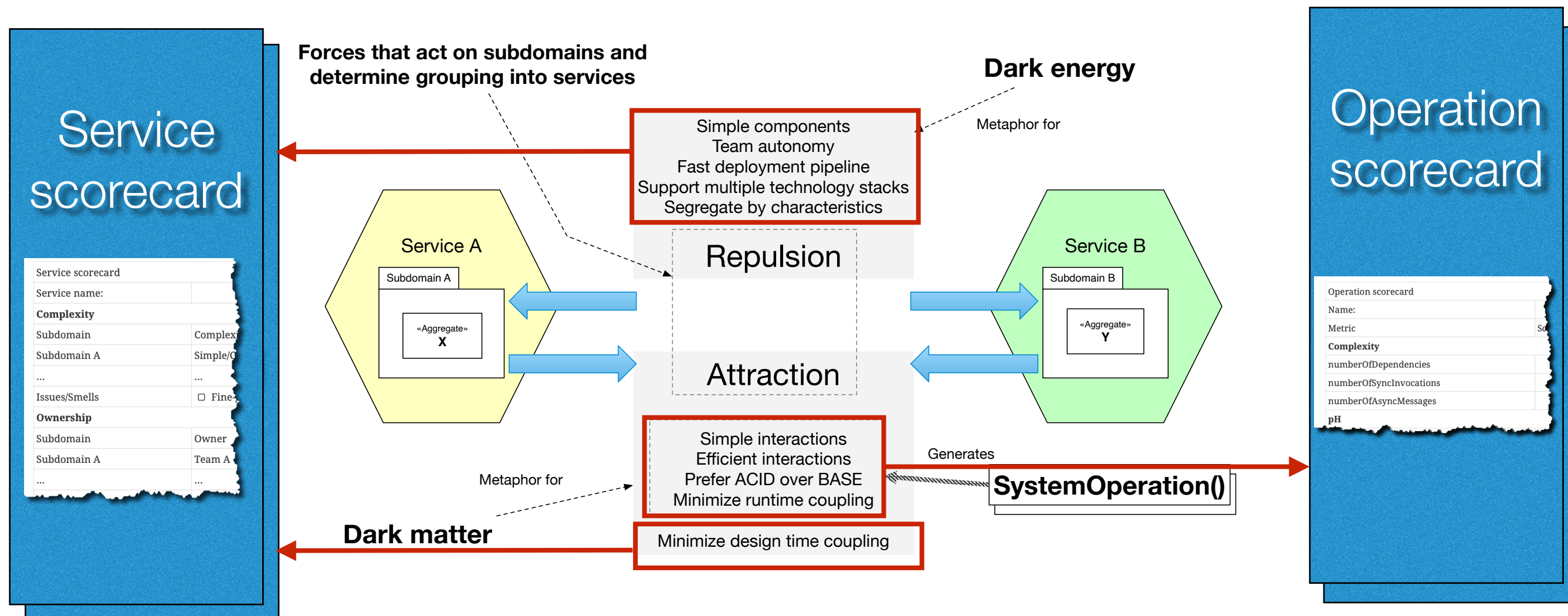


Agenda

- Overview of evaluating an architecture
- Evaluating services
- Evaluating operations

Evaluate architecture:
Identify unresolved dark
energy and dark matter forces

Evaluation => service and operation scorecards



Identifying potential vs. actual problems

Design-phase

Design artifacts

Predict
potential
problems

Deployment-phase

Development
“Environment”

Dev. sentiment
Metrics

Source
code repo

Commit
analysis

Production
“Environment”

Metrics

Find actual
problems

... identifying potential vs. actual problems

	Potential	Actual
Runtime coupling	Deep sequence diagrams	Latency and availability metrics
Design-time coupling	Various indicators such as API size, dependencies, ...	Git commits for multiple services for same issue
...	...	...

When to evaluate: evaluate and select design candidate for operation realization

3

Design services and operations

Rank system operations

For each operation:

Generate candidate designs

Group subdomains

Design collaborations

Evaluate candidates

Select candidate design

Operation scorecard

Service scorecards

Candidate 1

Candidate 2

Operation scorecard

Name:

Metric

Complexity

numberOfDependencies

numberOfSyncInvocations

numberOfAsyncMessages

pH

Operation scorecard

Name:

Metric

Complexity

numberOfDependencies

numberOfSyncInvocations

numberOfAsyncMessages

pH

Service scorecard

Service name:

Complexity

Subdomain

Subdomain A

...

Issues/Smells

Ownership

Subdomain

Subdomain A

...

Service scorecard

Service name:

Complexity

Subdomain

Subdomain A

...

Issues/Smells

Ownership

Subdomain

Subdomain A

...

When to evaluate: globally

- Designing each operation realization = series of locally good decisions
- But can result in a globally suboptimal architecture
- Requirements change overtime

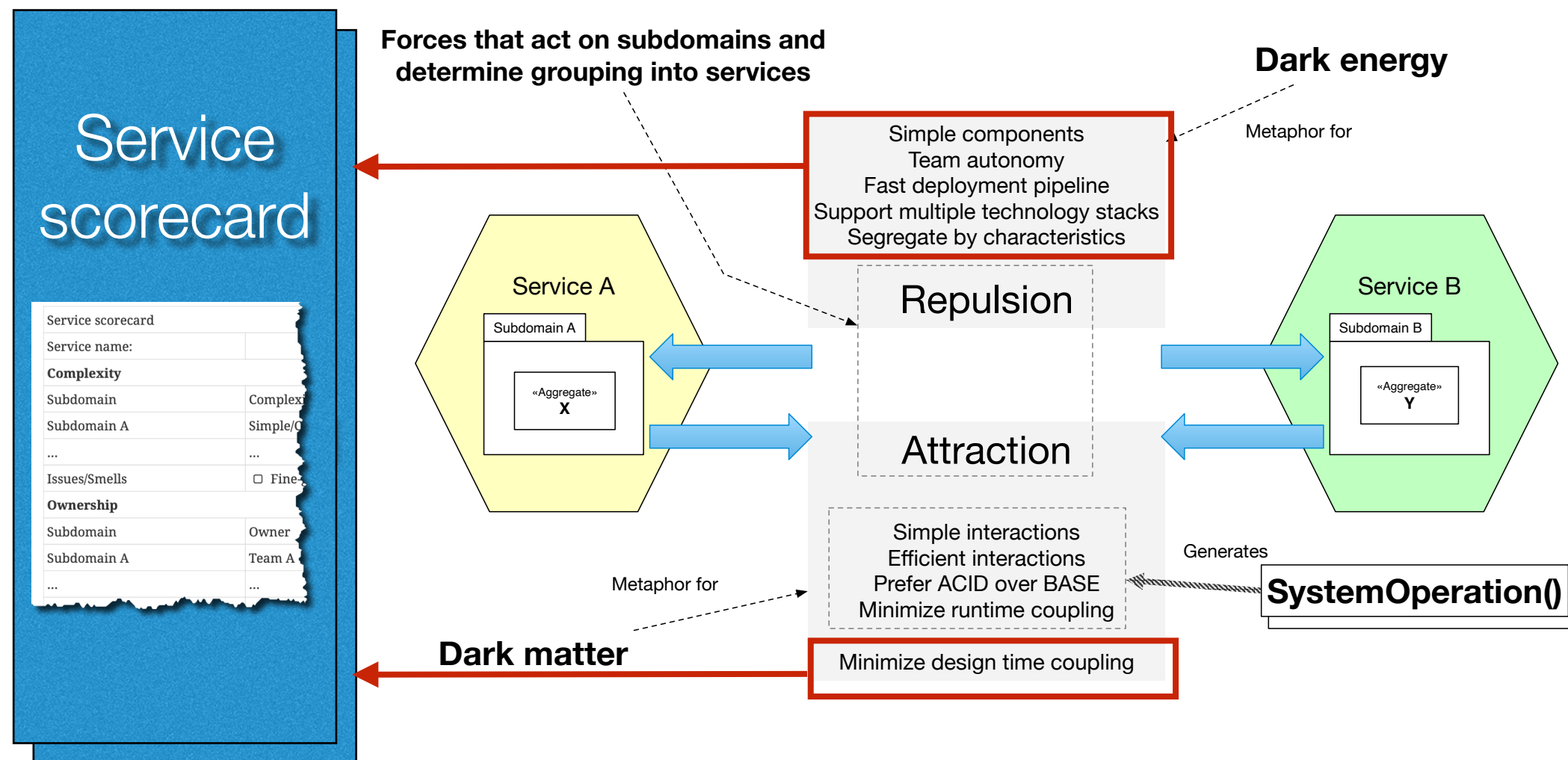
THEREFORE evaluate the architecture

- When first defining the architecture: e.g. Monolith -> microservices migration
- During the lifetime of the application
 - Implementing new features
 - On-going architectural evolution

Agenda

- Overview of evaluating an architecture
- Evaluating services
- Evaluating operations

Evaluating services



Evaluating service complexity

Service scorecard

Service name:

Complexity

Subdomain

Complexity

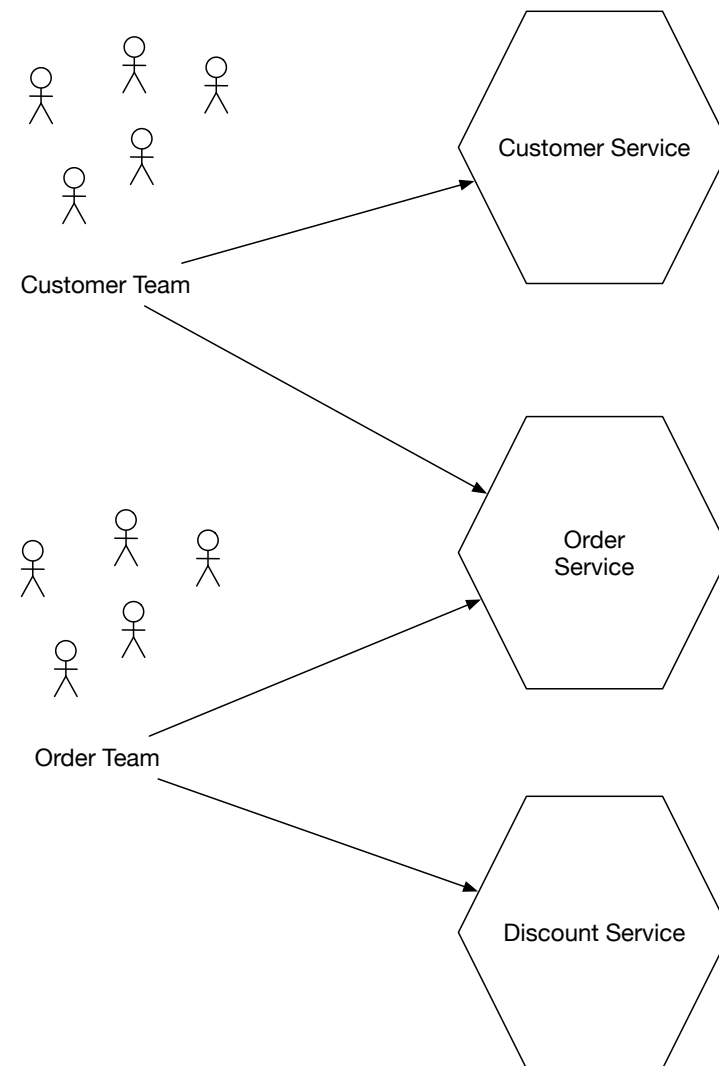
Subdomain A

Simple/Complicated/Complex

...

...

Evaluating team autonomy



Architectural smell:
Shared responsibility

Ideally: one team, one
service
⇒
Justify deviation

Team	Services	# Services
Customer	Customer Service Order Service	2
Order	Customer Service Discount Service	2

Service	Team	# Team
Customer Service	Customer	1
Order Service	Customer Order	2
Discount Service	Order	1

Evaluating team autonomy: survey teams!

Do you agree with the following statements?

“Complete their work without communicating and coordinating with people outside their team”

“Make large-scale changes to the design of their system without depending on other teams to make changes in their systems or creating significant work for other teams”

“We can and do deploy or release our application independently of other applications/services it depends on.”

Evaluating the deployment pipeline

Service scorecard

Service name:

Testability

Lead time	10 minutes
Subdomain	Locally testable
User Management	Yes
SMS Notifications	No
...	...

Evaluating the deployment pipeline

- ✦ Lead time:
 - ✦ Time from commit to deploy
 - ✦ Deployment pipeline execution time
 - ✦ Ideally < 15 minutes
- ✦ Testable locally?
 - ✦ Can the service be tested locally, e.g. developer laptop
 - ✦ Required

Architectural smell:

Sluggish service

Architectural smell:

Unverifiable

Evaluating technology stack upgradability

- Does any subdomain have dependencies that would complicate technology upgrades?
 - A dependency might not support newer versions of libraries, JDK, etc
- Service upgrade effort proportional # dependencies

Evaluating subdomain segregation

- mix of characteristics?

Service scorecard	
Service name:	
Business criticality	
Sub-domain	Business Criticality
A	Tier 1
...	...
Issues/Smells	☐ Mixed bag (mix of subdomains with different characteristics)
Regulated development	
Sub-domain	Regulation
A	SaMD
...	...
Issues/Smells	☐ Mixed bag
Cost-effective scaling	
1	

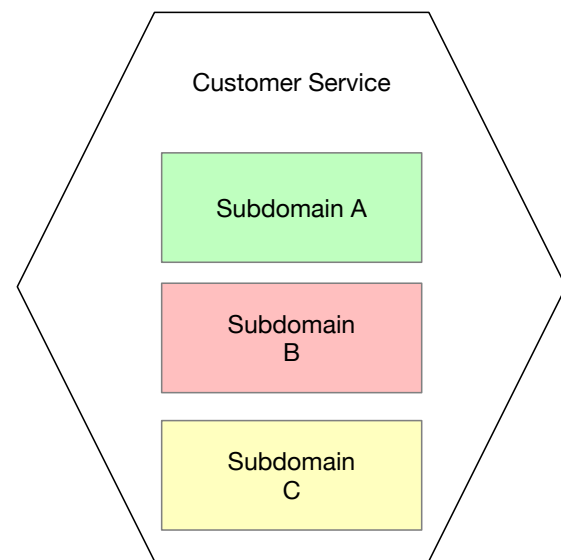
Sub-domain	Required resources
...	...
...	...
Issues/Smells	☐ Mixed bag

Architectural smell:

Mixed bag*

* mixed bag = collection of subdomains with different characteristics

Evaluating cost effective scaling



Ideally: a service's subdomains should have similar resource requirements

Customer Service				
Subdomain	Required resources			
	CPU	GPU	Memory	I/O
Subdomain A	...	...	...	...
Subdomain B	...	...	...	...
Subdomain C	...	...	...	...

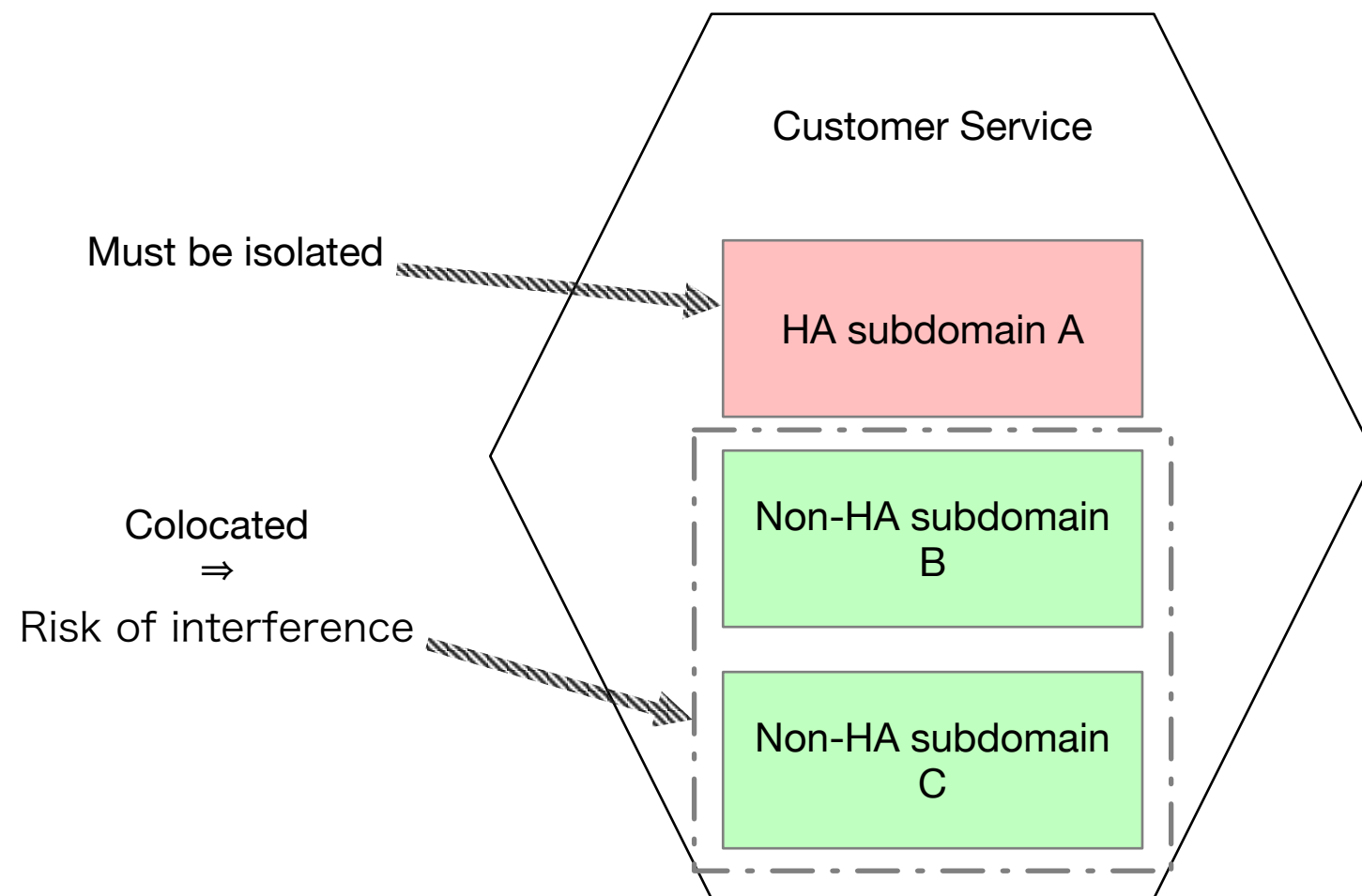


Service	# subdomains with incompatible resource needs
...	...

Architectural smell:

Money pit

Evaluating segregation by business criticality



Ideally: an HA subdomains should be separate/isolated from non-HA subdomains

Service with HA subdomains	Non-HA subdomains	# non-HA subdomains
...	...	...

Architectural smell:

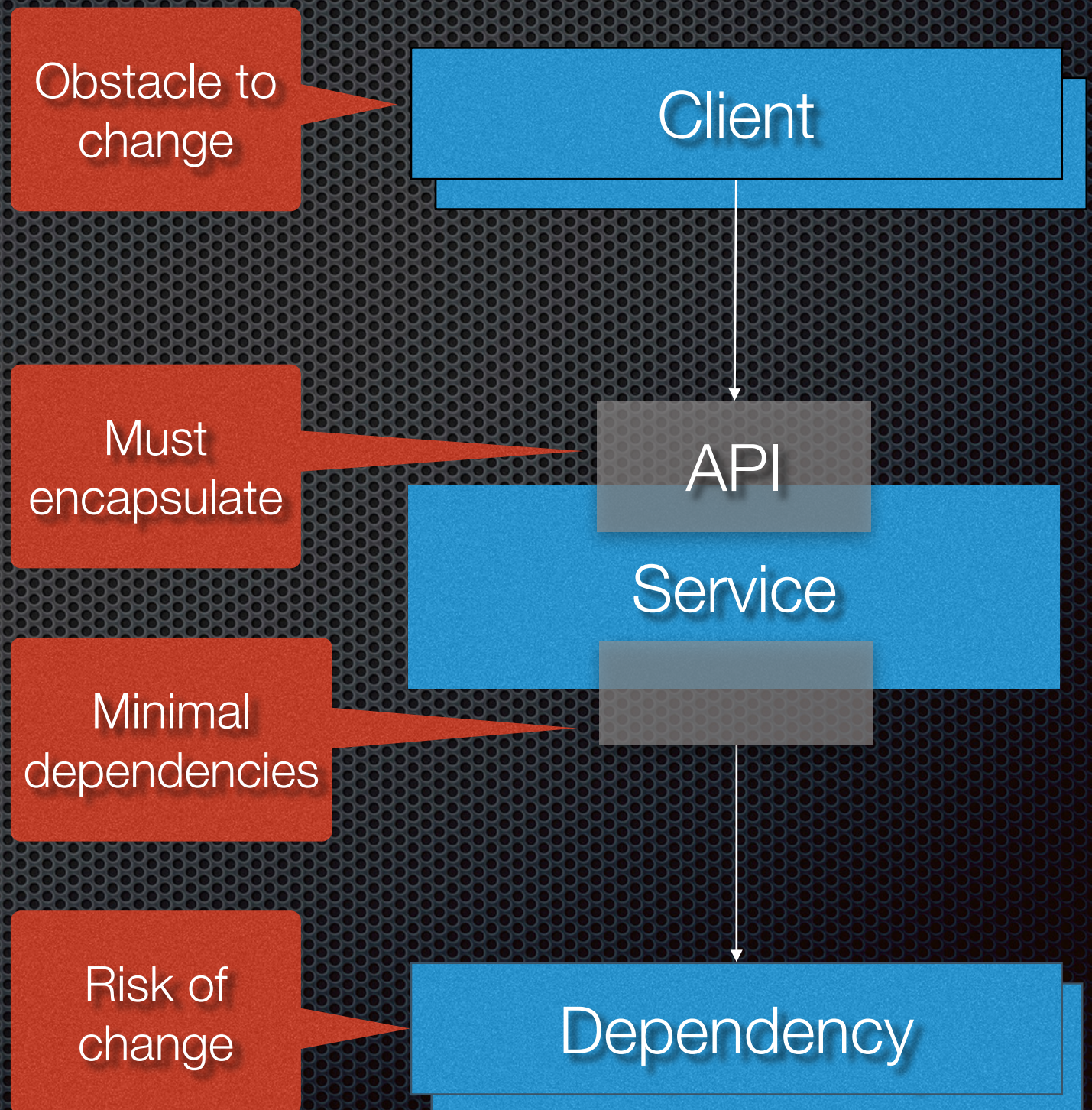
Mixed bag

Evaluating design-time coupling

Measure actual frequency
of lock step changes

OR

Assess the design



Evaluating design-time coupling: analyzing Git commits

Frequent lock-step changes indicate tight design-time coupling

JIRA issue
Implement X

Order Service
Git Repository

Git commit

Customer Service
Git Repository

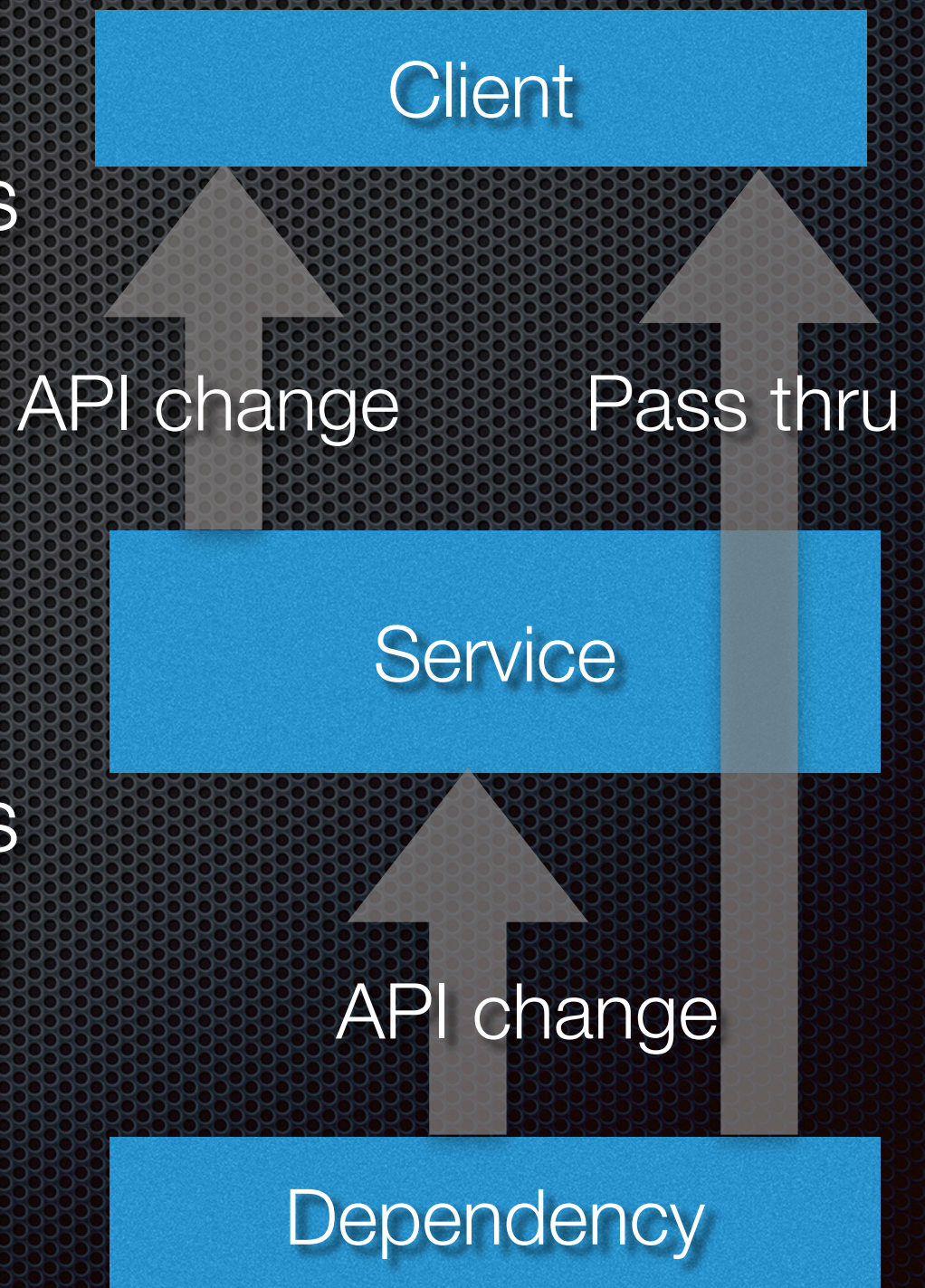
Git commit

Measure directly: frequency of breaking API changes

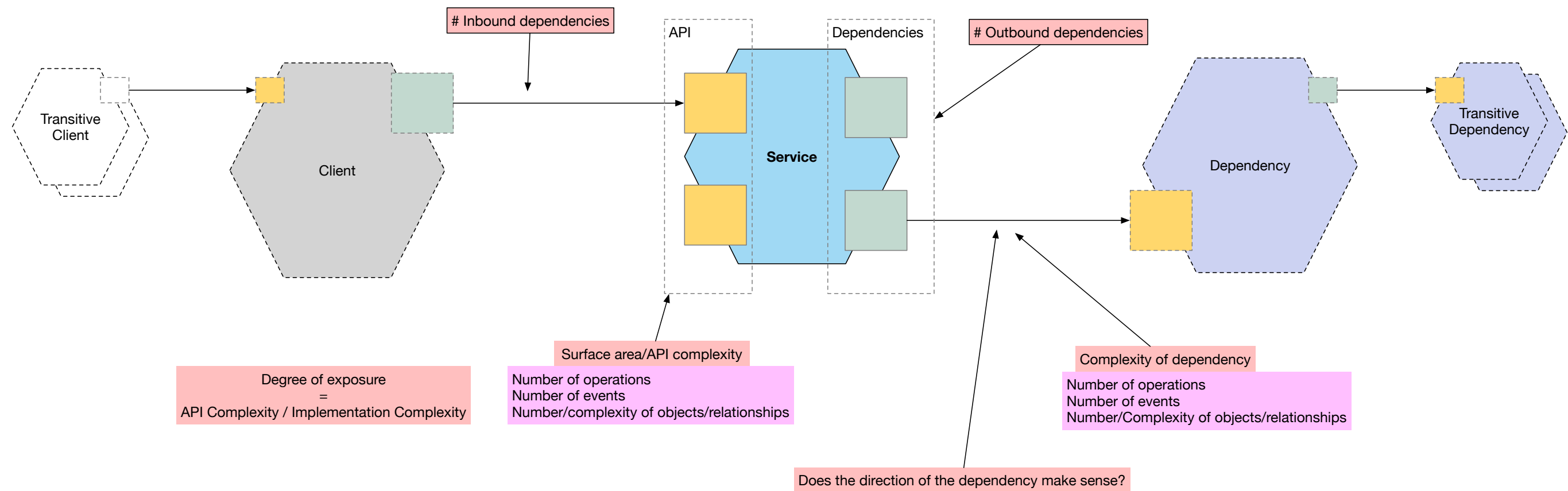
- Quantified:
 - Interval between breaking API changes
 - % commits
- Subjective:
 - Never
 - Rarely
 - ...
 - Often

Impacts
clients

Impacts
service



Evaluating design-time coupling: metrics

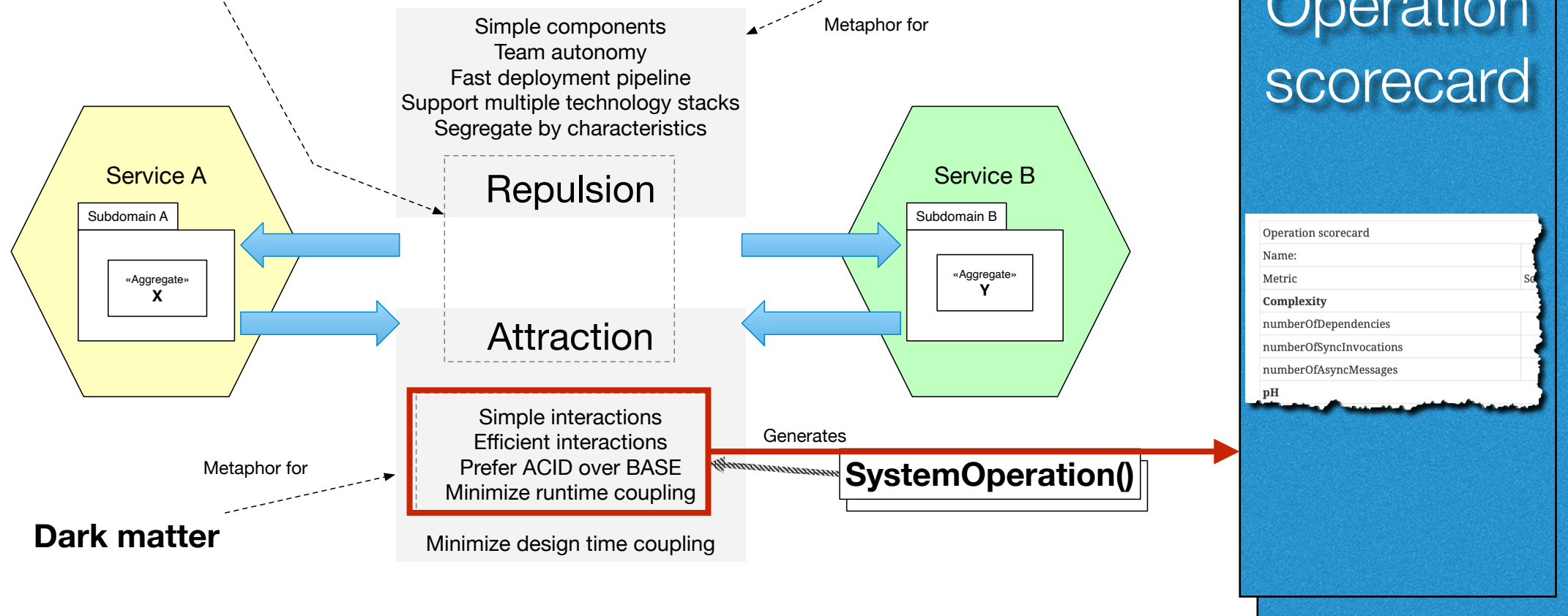


Agenda

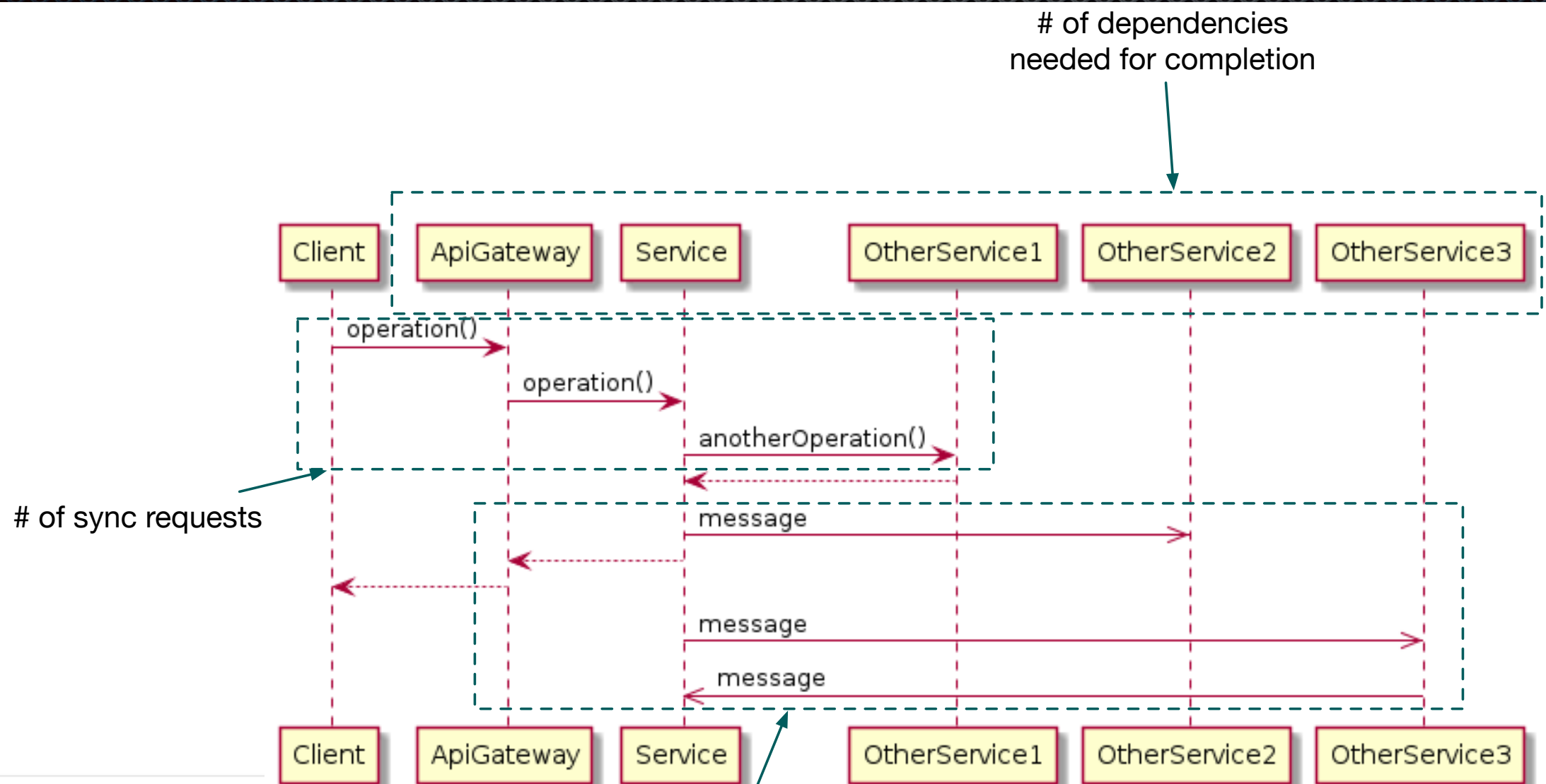
- Overview of evaluating an architecture
- Evaluating services
- Evaluating operations

Evaluation => service and operation scorecards

Forces that act on subdomains and determine grouping into services



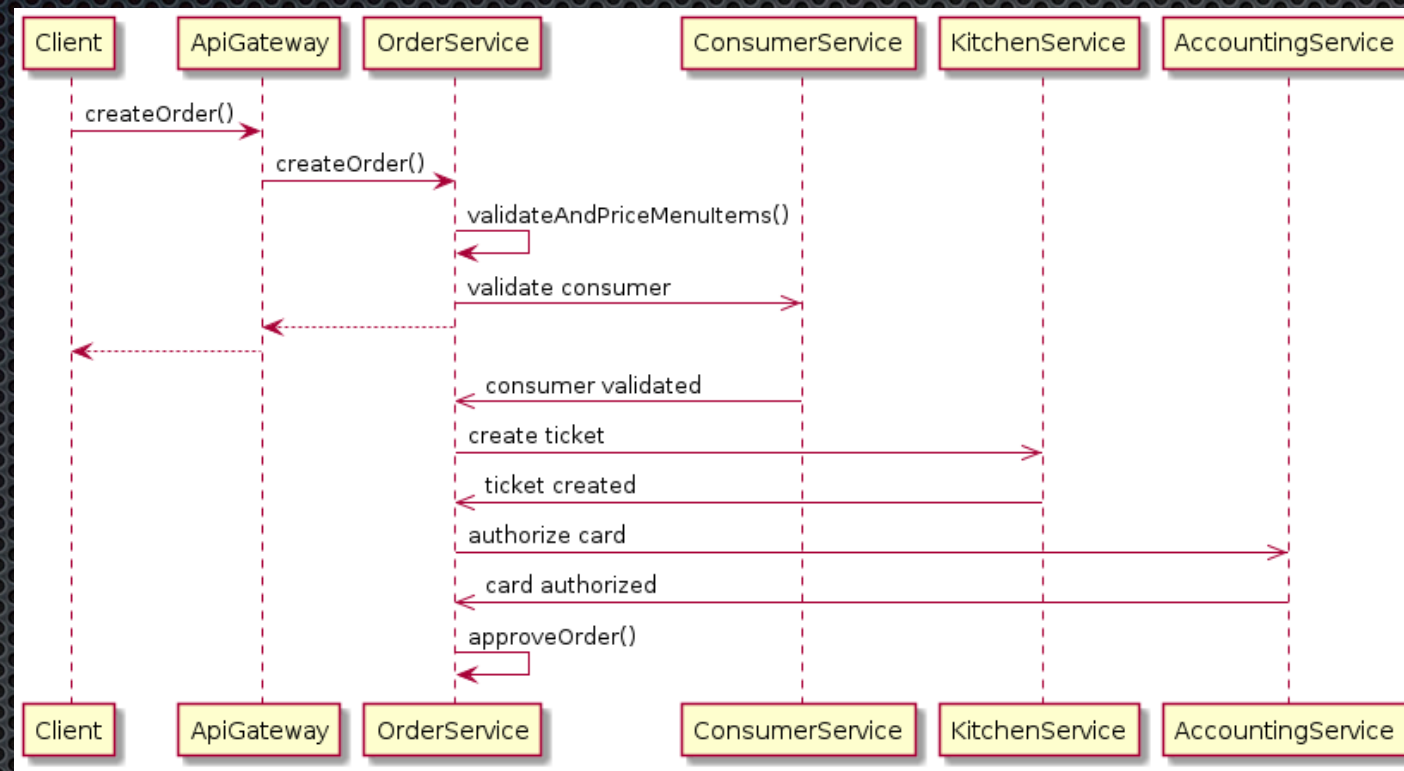
Evaluating interaction complexity



Operation scorecard

Name:	
Metric	Score
Complexity	
numberOfDependencies	
numberOfSyncInvocations	
numberOfAsyncMessages	

Evaluating interaction efficiency



Operation scorecard

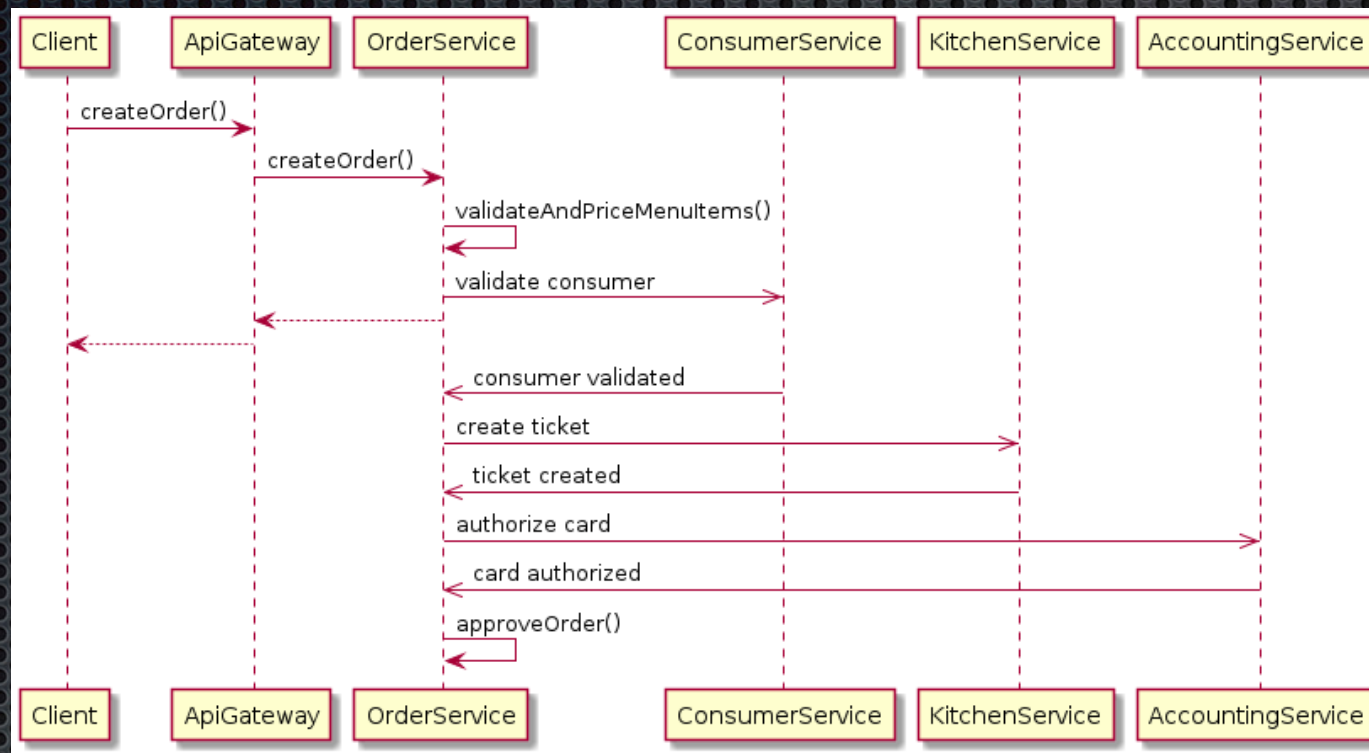
Name:

Efficiency

numberOfMessages

volumeOfData

Evaluating “ACIDity”



Operation scorecard

Name:

pH

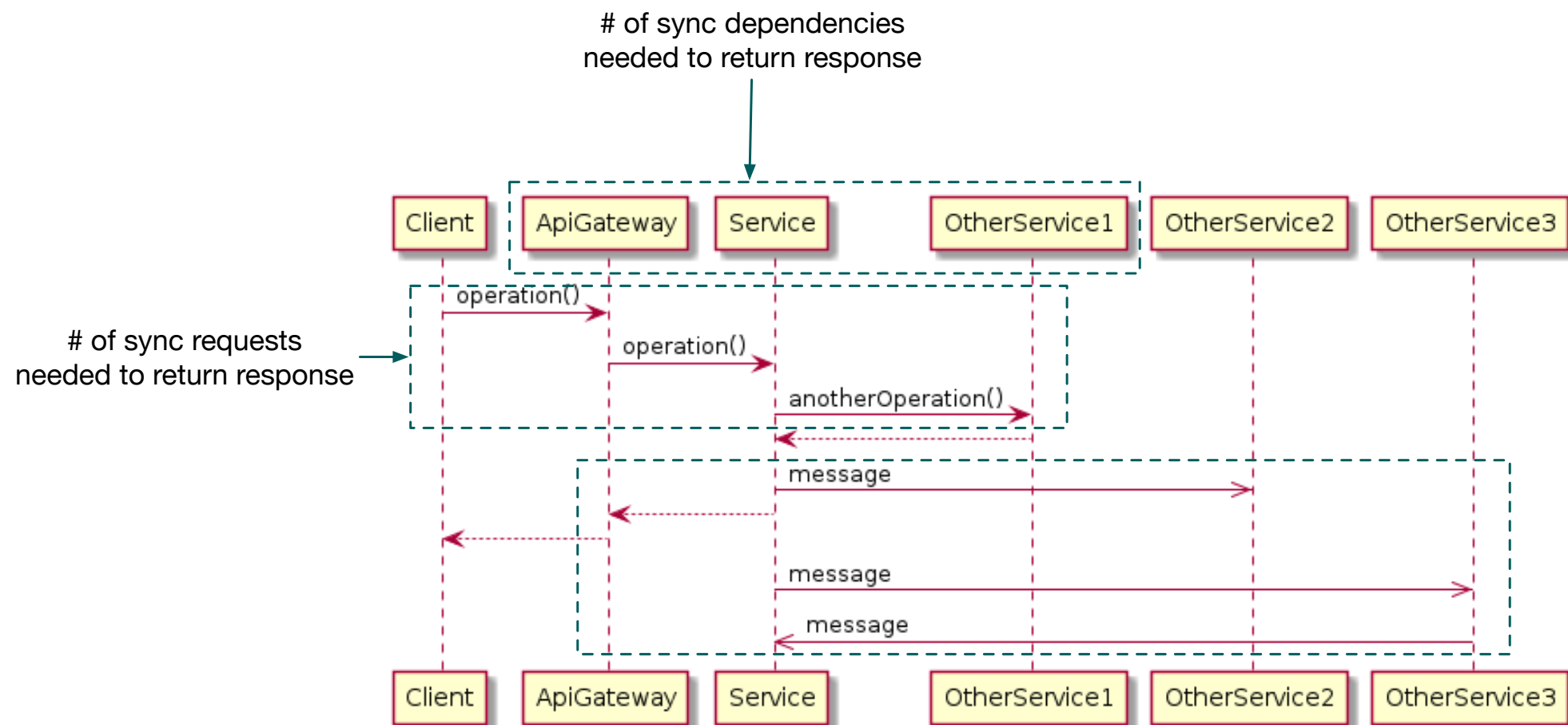
services updated

compensating transactions

services queries

CQRS views queried

Evaluating runtime coupling



Operation scorecard

Name:

Runtime coupling

numberOfSyncDependencies

numberOfSyncInvocations

maxSyncRequestDepth

Use endpoint latency and
availability metrics from the
running application

 [@crichardson](https://twitter.com/crichardson) chris@chrisrichardson.net

Questions?

adopt.microservices.io